City Hall Operational Case Study

From Rough Project Idea to Governed Library Standard

Case date: July 22, 2026
Workspace: Aptlantis Development Drive
Governance system: Aptlantis City Hall
Project: `render-manifest.crate`
New standard produced: Library Development Standard (LDS) v0.1.0
Agent: Claude
Outcome: Successful end-to-end project onboarding, governance-gap detection, standards creation, registration, and controlled handoff

---

Executive Summary

On July 22, 2026, a loosely formed idea for a Rust crate was introduced into the Aptlantis development workspace.

The initial idea was intentionally narrow: create a Rust library that could import structured TOML records and render them as sections, properties, tables, text, and evidence. The project was not fully designed, did not have final naming, and was not ready for implementation.

Claude was asked to onboard the idea into the development-drive ecosystem using the documentation and governance maintained by Aptlantis City Hall.

The agent successfully completed the full intake process:

1. It located and read the appropriate workspace-governance documents.
2. It classified the idea as a library-first Rust crate family.
3. It created the required project proposal and companion records.
4. It evaluated the existing delivery standards.
5. It determined that none correctly governed a library whose primary consumer was other code.
6. It rejected an inappropriate reuse of the Dataset Development Standard.
7. It followed the Standards Framework Development Standard to create a new Library Development Standard.
8. It registered the new standard in the relevant City Hall governance locations.
9. It applied LDS to the library portions of the proposed crate family.
10. It assigned the planned CLI and service surfaces to CTS and SIS respectively.
11. It stopped before implementation because the project remained at PPS readiness level `draft`.

The result is a real example of City Hall accomplishing its intended purpose from beginning to end.

The system did not merely help an agent write documents. It enabled an agent with limited prior workspace context to discover the correct process, identify a structural gap, extend the governance system through its own approved mechanism, and leave behind a complete and recoverable project record.

---

1. Background

Aptlantis City Hall is the governance and standards center for the Aptlantis workspace.

Its purpose is to explain:

- how projects begin,
- how the workspace is organized,
- which standards govern different artifact classes,
- how standards mature,
- how work is validated,
- and how future humans or agents recover enough context to continue safely.

The City Hall principle is not paperwork for its own sake. Its purpose is to reduce ambiguity and prevent project knowledge from existing only in one person's memory or one agent's conversation history.

The `render-manifest.crate` case provided an opportunity to test whether that system was sufficiently documented to guide a relatively less-contextual agent through a new-project workflow.

---

2. Initial Project Idea

The project began as a rough concept rather than a technical specification.

The desired capability was a Rust crate that could:

- import structured TOML records,
- validate their structure,
- interpret simple presentation intent,
- and render project records as sections, properties, tables, text, and evidence.

The project was deliberately not intended to become a general application framework, low-code platform, or arbitrary user-interface language.

The project proposal ultimately captured the concept as:

> A manifest-driven presentation engine: a Rust toolkit that turns structured TOML records containing content and presentation intent into a validated, renderer-neutral document model that React, static HTML, Markdown, a CLI, or another Rust service can consume.

The idea was useful but incomplete. At intake time:

- no source code existed,
- the crate names were not final,
- the public API was speculative,
- the governing delivery standard had not been identified,
- and the project had not reached implementation readiness.

The WGS lifecycle state was therefore recorded as `concept`, while the PPS readiness level was recorded as `draft`.

---

3. Agent Entry and Orientation

Claude entered the project through the workspace's documented governance path rather than relying on prior conversational familiarity.

The project-specific `AGENTS.md` established a read-first sequence:

1. parent workspace instructions,
2. the project manifest,
3. the project proposal,
4. the project README,
5. the library interface note,
6. and the applicable governing standards.

This sequence gave the agent a deterministic orientation path.

The file also made the project's current state unambiguous:

> This is a PPS proposal at readiness level `draft`, not an implemented crate. There is no source code yet. Do not scaffold a Cargo workspace, add crates, or begin broad implementation from this state alone.

This was important because it separated project onboarding from project implementation. The agent was authorized to document, classify, and govern the idea, but not to begin building it prematurely.

---

4. Project Classification

Claude classified the proposal as a Rust crate family with three potential delivery shapes:

Library crates

- `manifest-render-core`
- `manifest-render-schema`

- `manifest-render-html`

These crates would primarily be consumed by other code.

Command-line crate

- `manifest-render-cli`

This would provide a human- and automation-facing command interface.

Service crate

- `manifest-render-axum`

This was identified as a possible future HTTP service but deliberately deferred from the first release.

This classification mattered because City Hall standards are assigned according to the consumption and delivery shape of an artifact, not merely the language or repository containing it.

---

5. Discovery of the Governance Gap

The existing City Hall delivery standards covered several well-defined artifact classes:

- CTS covered command-line tools.
- SIS covered services and infrastructure.
- WDS covered websites.
- DRS covered desktop applications and releases.
- DDS covered datasets.

However, none of those standards governed a bare library whose primary product was code consumed by other code.

The core rendering crates did not have a command contract, so CTS was not sufficient.

They did not run continuously as services, so SIS was not appropriate.

They were not websites, desktop applications, or datasets.

This left a genuine delivery-standard gap.

The project proposal recorded the conclusion directly:

> `manifest-render-core`, `manifest-render-schema`, and `manifest-render-html` are pure libraries with no CLI or service entry point of their own; neither CTS nor SIS named this consumption shape, so a new standard was proposed and adopted.

---

6. Rejection of an Incorrect Existing Standard

Claude considered whether the Dataset Development Standard could be adapted to govern the project.

It rejected that option.

DDS required concerns such as:

- collected-data provenance,
- licensing,
- dataset splits,
- corpora,
- and dataset-release integrity.

Those requirements did not correspond to the delivery concerns of a Rust library.

The project proposal documented the reasoning:

> DDS was considered and rejected as a governing standard because its required artifacts do not map onto a rendering library, and repurposing an idle standard for an unrelated domain would blur the responsibility boundaries maintained by WGS and SFDS.

This was an important success condition.

The agent did not force the project into the nearest available category merely to finish the paperwork. It preserved the semantic boundaries of the standards suite.

---

7. Creation of the Library Development Standard

After determining that the gap was real, Claude followed the Standards Framework Development Standard to create a new delivery standard.

The result was the **Library Development Standard**, abbreviated **LDS**.

LDS was created at version `0.1.0` with `draft` maturity. Its manifest defines its scope as libraries, packages, crates, and SDKs whose primary deliverable is code consumed by other code.

The new standard governs:

- public API surface definition,
- stability levels,
- versioning and breaking-change policy,
- minimum supported runtime or toolchain constraints,
- extension contracts,
- and known-consumer tracking.

It explicitly does not govern:

- CLI command behavior,
- service lifecycle,
- website deployment,
- desktop packaging,
- dataset provenance,
- workspace placement,
- or project mission and success criteria.

This preserved clean responsibility boundaries between LDS and the existing City Hall standards.

---

8. LDS Document Suite

Claude created the initial LDS suite rather than producing only a single specification file.

The suite included:

- `README.md`
- `Library Development Standard.md`
- `LDS.manifest.toml`
- `Adoption-Guide.md`
- `Validation-Checklist.md`
- `CHANGELOG.md`
- `templates/Library-Interface-Note.md`

The LDS README identifies the standard's role as filling the delivery gap between commands, services, websites, desktop applications, and datasets.

The changelog records the initial creation of the suite, the definition of library stability levels, the scope boundary against adjacent standards, and the selection of `render-manifest.crate` as the first candidate adopter.

---

9. Library Stability Model

LDS introduced four evidence-based stability levels:

Level	Meaning
`experimental`	The public API is not fixed and breaking changes may occur without notice.
`interface-stable`	The public surface is stable enough for an independent second consumer.

Level	Meaning
`versioned`	A documented versioning policy is enforced and breaking changes require recorded version transitions.
`reference`	The library has multiple tracked consumers and a maintained history that includes at least one breaking change.

These levels are defined in the LDS specification.

The model avoids treating stability as a purely subjective declaration.

Higher maturity depends on actual interface use, consumer tracking, and demonstrated change management.

---

10. Adopter-Facing Interface Record

LDS adds one focused adopter artifact: `Library-Interface-Note.md`.

The template records:

- public API surface,
- stability level,
- versioning and breaking-change policy,
- extension contracts,
- known consumers,
- companion crates,
- and known gaps.

For `render-manifest.crate`, the initial note correctly states that the public API does not yet exist.

It records the expected future model—such as a `Document` and `Block` representation, parse and validate entry points, JSON normalization, and a proposed `Renderer` trait—while clearly marking the interface as speculative and experimental.

It also records that the planned CLI is governed by CTS and the deferred service by SIS.

---

11. Project Governance Produced

Claude created the complete planning and governance set for the project:

- `Project-Proposal.md`
- `render-manifest.crate.manifest.toml`
- `AGENTS.md`

- `Project-README.md`
- `Library-Interface-Note.md`

The project README records:

- the purpose and boundaries,
- governing standards,
- current lifecycle state,
- proposed architecture,
- intended repository structure,
- expected first workflow,
- known gaps,
- and handoff information.

The first implementation workflow was defined as:

> parse a manifest → validate it → produce a normalized JSON document tree → render it to HTML.

No `Cargo.toml`, `src` directory, or crate implementation was created.

---

12. Preservation of Scope Boundaries

The project proposal captured explicit permanent non-goals.

Rust may own:

- parsing,
- correctness,
- interpretation,
- validation,
- transformation,
- and normalized output.

Rust must not own the final visual appearance.

The manifest language must describe documents and records rather than applications.

It must not expand to express:

- authentication,
- buttons,
- mutations,
- modal behavior,
- routing,
- state machines,

- or arbitrary client-side logic.

These boundaries directly preserve the original intent of the idea: a small, opinionated rendering library rather than a never-ending framework.

The project's own failure criteria treat violation of those boundaries as project failure, not merely an undesirable future direction.

---

13. Controlled Multi-Standard Assignment

The case demonstrated that standards can be applied at the crate or delivery-surface level.

The resulting governance map was:

```
manifest-render-core      → LDS
manifest-render-schema    → LDS
manifest-render-html      → LDS
manifest-render-cli       → CTS
manifest-render-axum      → SIS
```

This avoided assigning one broad standard to the entire crate family when its components had different delivery contracts.

The LDS specification explicitly permits this layered approach: a core library can be governed by LDS while companion command and service crates are governed by CTS and SIS.

---

14. Registration and City Hall Integration

The work did not stop at creating isolated files.

The new standard was registered in the relevant governance locations.

The handoff notes report that Claude:

- created the LDS suite under City Hall,
- added LDS to the WGS Governance Responsibility Matrix,
- listed LDS in the SFDS applicability material,
- and identified `render-manifest.crate` as the first candidate adopter.

The project remained staged under `D:\.zoning`, which is the intentional paperwork and intake location before implementation begins.

The agent did not mistake that staging directory for the permanent code root.

15. Deliberate Stopping Point

A major success of the case was not only what the agent completed, but what it refused to do prematurely.

The project remained:

- lifecycle state: `concept`
- PPS readiness: `draft`
- implementation status: not started

The agent was instructed not to scaffold a Cargo workspace until the proposal was promoted to `ready`.

It obeyed that restriction.

The handoff's next safe action was not "continue automatically." It was to begin Phase 1 only when the maintainer was ready, using real implementation work to refine the speculative interface note and test LDS in practice.

This demonstrated that City Hall can support productive autonomy without granting unlimited implementation authority.

---

16. Why the Case Matters

16.1 The workspace carried the context

Claude was not the maintainer's primary or most context-rich agent.

The successful result therefore did not depend mainly on accumulated conversational memory.

The necessary context had been transferred into:

- documented read paths,
- standards,
- manifests,
- responsibility boundaries,
- lifecycle states,
- templates,
- validation checklists,
- and project-specific instructions.

The workspace itself had become capable of orienting the agent.

16.2 Documentation functioned as compressed context

Without City Hall, onboarding the project would likely have required repeatedly explaining:

- directory placement,
- project proposal expectations,
- standard selection,
- lifecycle states,
- non-goals,
- manifest conventions,
- and stopping conditions.

Because these were already documented, the agent could recover them through targeted reading instead of requiring a full conversation window of custom explanation.

16.3 The governance system detected its own missing capability

The system did not merely classify a known project type.

It encountered an artifact class it did not cover and used its own standard-development process to extend itself.

That is a materially different capability from static documentation.

16.4 The extension was controlled

The new standard was:

- narrow in scope,
- clearly separated from adjacent standards,
- versioned,
- assigned draft maturity,
- given an adoption guide,
- given a validation checklist,
- and tested immediately against a real candidate adopter.

The system extended itself without discarding its boundaries.

16.5 The process produced a recoverable handoff

A future human or agent can determine:

- what the project is,
- why it exists,
- what is in and out of scope,
- which standards govern each part,
- what documents were created,

- why LDS was necessary,
- why DDS was rejected,
- what remains speculative,
- and what action is safe next.

The project no longer depends on reconstructing the original conversation.

---

17. Observed Success Criteria for City Hall

This case provides evidence that City Hall can successfully:

- orient an agent with limited prior context,
- route an idea through the correct project-creation process,
- distinguish planning from implementation,
- classify mixed project-delivery surfaces,
- detect a missing standards category,
- avoid misusing an adjacent standard,
- create a new standard through SFDS,
- register the new standard,
- apply it to a first candidate adopter,
- record unresolved questions,
- enforce readiness gates,
- and produce a durable handoff.

The process succeeded from initial concept intake through governance completion without uncontrolled implementation.

---

18. Remaining Limitations

This case validates the workflow, but it does not yet prove every aspect of LDS.

LDS is still draft v0.1.0.

`render-manifest.crate` is its first candidate adopter, and no source code exists yet.

The adopter template has not yet been validated against:

- a completed public API,
- a second independent library,
- multiple real consumers,
- or an actual breaking-change cycle.

The project proposal explicitly recognizes that LDS may require refinement once implementation begins and the standard encounters real library-design pressures.

Therefore, this case demonstrates successful governance creation and adoption, not final proof that LDS is mature or complete.

---

19. Conclusion

The `render-manifest.crate` onboarding is a concrete operational success for Aptlantis City Hall.

A rough project idea entered the workspace without a final design or an obvious governing delivery standard.

The agent:

- found the correct documentation,
- created the project's planning records,
- respected lifecycle and readiness boundaries,
- identified a genuine standards gap,
- rejected an incorrect substitute,
- used SFDS to create LDS,
- registered the standard,
- applied the correct standards to each proposed crate,
- and stopped before unauthorized implementation.

The result demonstrates that City Hall is no longer only a collection of governance documents.

It is functioning as a **system for producing governed systems**.

The strongest evidence is not that the agent generated a large amount of documentation. It is that the agent made the same kinds of distinctions the workspace was designed to preserve:

- proposal versus implementation,
- library versus command,
- command versus service,
- correct fit versus convenient fit,
- explicit boundary versus silent scope growth,
- and autonomous progress versus unauthorized action.

This case should be retained as the first formal example of City Hall successfully guiding a project from an informal idea through project intake, governance-gap discovery, standards creation, registration, first adoption, and safe handoff.

Case Study Status

Result: Successful
City Hall behavior demonstrated: End-to-end project onboarding and controlled governance extension

New governance capability: Library Development Standard
First candidate adopter: `render-manifest.crate`
Implementation authorized: No
Next decision: Maintainer review and promotion from PPS `draft` to `ready` before Phase 1 implementation